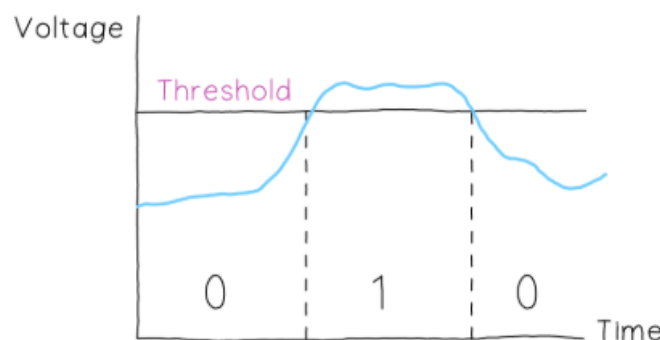


All about Qubits

Notion of Qubit

- Quantum computers are a new types of technology that can solve some problems much faster than even most powerful classical computers.
- Regular computers use *bits* to store information. A *bit* can be only 0 or 1. This is usually represented by a physical state, like **low voltage = 0** & **high voltage = 1**.



- Quantum computers use *qubits* (*quantum bits*) instead of regular bits. Like bits, qubits are also based on physical systems, but they behave according to the rules of quantum physics.
- Quantum computing means manipulating qubits to solve problems.
- To understand how quantum computers work, we need three basic ideas:
 - i. A mathematical way to represent the final state of a qubit
 - ii. A method to measure a qubit and find its state
 - iii. A way to change the qubit's state to perform computations

Mathematical Representation of Qubit

- Quantum computing mainly uses linear algebra. A qubit is represented mathematically as a **column vector with two elements**.
- The two basic qubit states correspond to classical 0 & 1.

$$\text{qubit state 0} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \text{qubit state 1} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

- To make writing easier, quantum computing uses Dirac notation; *bra-ket notation*.
 - **ket** $\Rightarrow$ A qubit state and is written like **$|0\rangle$** or **$|1\rangle$**
 - Sometimes, we write $|\psi\rangle$ to represent a qubit in any general state

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

- For every ket, there is bra.
 - **bra** $\Rightarrow$ Written like **$\langle 0|$** or **$\langle 1|$**
 - It is basically row-vector version of the ket

$$\langle 0| = (1 \ 0), \quad \langle 1| = (0 \ 1).$$

- The states **$|0\rangle$** and **$|1\rangle$** very important because they form a basis.
- A basis means any other qubit state can be created by combining these two states.
- These two states are also orthogonal $\Rightarrow$ Their inner product (dot product) is zero.

$$\begin{aligned} \langle 0||1\rangle &= \langle 0|1\rangle = (1 \ 0) \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\ &= 1 \cdot 0 + 0 \cdot 1 \\ &= 0. \end{aligned}$$

- When we combine a bra and a ket, we get a bra-ket expression, such as **$\langle 0|1\rangle$** .
- The states **$|0\rangle$** and **$|1\rangle$** form the computational basis, which is the most common way to represent the qubits.
- They are also normalized, meaning their vector length is 1.

$$\begin{aligned}\sqrt{\langle 1|1\rangle} &= \sqrt{(0 \ 1) \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}} \\ &= \sqrt{0 \cdot 0 + 1 \cdot 1} \\ &= 1.\end{aligned}$$

- When basis vectors are both orthogonal and normalized, the set is called an **orthonormal basis**.

Superposition

- $|0\rangle$ and $|1\rangle$ are not the only possible states for a qubit.
- Qubits can exist in a superposition state, somewhere *between* $|0\rangle$ and $|1\rangle$.
- Mathematically, the state of a qubit in superposition is a linear combination of the basis state.
 - α and $\beta \Rightarrow$ Complex numbers

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

- $*$ $\Rightarrow$ Complex conjugate

$$\alpha\alpha^* + \beta\beta^* = 1$$

- α and $\beta \Rightarrow$ Amplitudes or Probability amplitudes, carries info about relative strength of $|0\rangle$ and $|1\rangle$ in the state.

A common misunderstanding $\Rightarrow$ A qubit in superposition is in a superposition of states, which is false because a qubit is only ever in one state, it is just that sometimes that state may be a linear combination of the basis states.

- Suppose we have two states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \text{and} \quad |\phi\rangle = \gamma|0\rangle + \delta|1\rangle$$

- Taking inner product between them $\langle\psi|\phi\rangle$.

- Computing bra of $|\Phi\rangle$
 - Taking bra of each basis state one at a time

$$\langle\phi| = \gamma^*\langle 0| + \delta^*\langle 1| = (\gamma^* \quad \delta^*).$$

- The inner product tells us about the overlap/similarity between two states.
- The result of the inner product = Complex number
 - If 0 $\Rightarrow$ Two states are orthogonal
 - If 1 $\Rightarrow$ Two states are same and normalized
- Ways to compute the inner product:
 - Writing the matrices:

$$\langle\phi|\psi\rangle = (\gamma^* \quad \delta^*) \begin{pmatrix} \alpha \\ \beta \end{pmatrix}.$$

- Bra-ket notation:

$$\begin{aligned} \langle\phi|\psi\rangle &= (\gamma^*\langle 0| + \delta^*\langle 1|) \cdot (\alpha|0\rangle + \beta|1\rangle) \\ &= \gamma^*\alpha\langle 0|0\rangle + \delta^*\alpha\langle 1|0\rangle + \gamma^*\beta\langle 0|1\rangle + \delta^*\beta\langle 1|1\rangle. \end{aligned}$$

Exercise

1. Evaluate the inner product $\langle\psi|\Phi\rangle$ and express the results in terms of amplitudes $\alpha, \beta, \gamma, \sigma$.

- The orthogonality of the basis state means:
 - $\langle 0|0\rangle = \langle 1|1\rangle = 1$
 - $\langle 1|0\rangle = \langle 0|1\rangle = 0$
- Thus, we obtain:

$$\langle\phi|\psi\rangle = \gamma^*\alpha + \delta^*\beta.$$

2. Verify that superposition state is normalized.

$$|\psi\rangle = \frac{3}{5}|0\rangle - \frac{4}{5}e^{\frac{\pi}{6}i}|1\rangle$$

- We take inner product of the state with itself and check if the result is 1.

$$\begin{aligned}\langle\psi|\psi\rangle &= \left(\frac{3}{5}\langle 0| - \frac{4}{5}e^{-\frac{\pi}{6}i}\langle 1|\right) \left(\frac{3}{5}|0\rangle - \frac{4}{5}e^{\frac{\pi}{6}i}|1\rangle\right) \\ &= \frac{9}{25} + \frac{16}{25} \\ &= 1.\end{aligned}$$

Measurement Outcome Probabilities

- To get meaningful result from a quantum computation, we must measure the qubits at the end.
- Measurement is probabilistic.
- A qubit in a superposition state is only ever observed as either **|0⟩** or **|1⟩**. The probability of each outcome is determined by its amplitude (α or β).

$$\begin{aligned}\text{Prob}(\text{measure and observe } |0\rangle) &= |\alpha|^2, \\ \text{Prob}(\text{measure and observe } |1\rangle) &= |\beta|^2.\end{aligned}$$

- $|\alpha|^2$ or $|\beta|^2 \Rightarrow$ mode squared $\Rightarrow$ Product of complex number and its conjugate; $|\alpha|^2 = \alpha\alpha^*$
- For a valid qubit, the sum of these probabilities must be 1, making quantum states normalized, i.e. $|\alpha|^2 + |\beta|^2 = 1$
- We map the measurement outcomes to classical bits:
 - $|0\rangle$ to classical "0"
 - $|1\rangle$ to classical "1"
- A single measurement gives only one bit of information, revealing a classical outcome.
- We must repeat the measurement many times to estimate the probabilities and determine the original quantum state.

Exercise

1. Is this state normalized? If so, what is the probability of observing the qubit in the state $|1\rangle$ after measuring it?

$$|\psi\rangle = \frac{1}{2}|0\rangle - \frac{\sqrt{3}i}{2}|1\rangle.$$

- Yes, the state is normalized, because:

$$\left|\frac{1}{2}\right|^2 + \left|\frac{\sqrt{3}i}{2}\right|^2 = 1$$

- The probability of observing it in $|1\rangle$ is:

$$-\frac{\sqrt{3}i}{2} \times \frac{\sqrt{3}i}{2} = \frac{3}{4}.$$

Operations on qubit states

- To manipulate qubit states, during a quantum algorithm, mathematical operations are used to transform one state vector into another.
- Vector transformation:
 - A qubit state $|\psi\rangle$ is modified into a new state $|\psi'\rangle$ using a 2X2 matrix, U :

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \rightarrow |\psi'\rangle = \alpha'|0\rangle + \beta'|1\rangle.$$

$$|\psi'\rangle = U|\psi\rangle.$$

- Unitary Matrices:

- Quantum operations must use unitary matrices to ensure the state remains normalized.
- The total probability of all measurement outcomes must still sum to 1; $|\alpha'|^2 + |\beta'|^2 = 1$
- The defining property of a unitary matrix:

$$UU^\dagger = I$$

- $I = 2 \times 2$ identity matrix

Exercise

Exercise 1.1.4. Suppose we have a qubit in state

$$|\psi\rangle = \frac{1}{2}|0\rangle - \frac{\sqrt{3}i}{2}|1\rangle,$$

and we want to apply the operation

$$U = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}.$$

Compute the state of the qubit after applying U . Then, compute the measurement outcome probabilities of 0 and 1, and verify that the state is normalized

▼ *Solution.*

$$U|\psi\rangle = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \\ \frac{-\sqrt{3}i}{2} \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{3}}{2} \\ \frac{-i}{2} \end{pmatrix} = \frac{\sqrt{3}}{2}|0\rangle - \frac{i}{2}|1\rangle$$

The probabilities of outcomes 0 and 1 are $3/4$ and $1/4$ respectively. Since they sum to 1, we know that the state is normalized. $\square$

Quantum computation in a nutshell

- Everything is linear algebra: qubit states are linear combinations of basis vectors $|0\rangle$ and $|1\rangle$ and operations on them are performed by matrices.

- The coefficients of those linear combinations tell us about the likelihood of a qubit being in one of those two states after measurement.
- Preparing states, performing operations, and taking measurements are the building blocks of all quantum algorithms.

CODERCISE 1.1.1 Normalization of Quantum States

In this codercise, you are given an unnormalized vector

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 \neq 1.$$

We can turn this into an equivalent, valid quantum state $|\psi'\rangle$ by *normalizing* it. Your task is to complete the function `normalize_state` so that, given α and β , it normalizes this state to

$$|\psi'\rangle = \alpha'|0\rangle + \beta'|1\rangle, \quad |\alpha'|^2 + |\beta'|^2 = 1.$$

```

1  # Here are the vector representations of |0> and |1>, for convenience
2  ket_0 = np.array([1, 0])
3  ket_1 = np.array([0, 1])
4
5
6  def normalize_state(alpha, beta):
7      # Create the initial unnormalized vector
8      state = np.array([alpha, beta])
9
10     # Calculate the norm of the vector
11     norm = np.linalg.norm(state)
12
13     # Divide the original state by the norm to normalize it
14     normalize_state = state / norm
15
16     return normalize_state
17

```

CODERCISE 1.1.2 Inner Product & Orthonormal Bases

Complete the `inner_product` function below that computes the inner product between two arbitrary states. Then, use it to verify that $|0\rangle$ and $|1\rangle$ form an **orthonormal basis**, i.e., the states are normalized and orthogonal.


```

1 def inner_product(state_1, state_2):
2     """Compute the inner product between two states.
3
4     Args:
5         state_1 (np.array[complex]): A normalized quantum state vector
6         state_2 (np.array[complex]): A second normalized quantum state vector
7
8     Returns:
9         complex: The value of the inner product <state_1 | state_2>.
10    """
11
12    # COMPUTE AND RETURN THE INNER PRODUCT
13
14    return np.dot(np.conj(state_1), state_2)
15
16
17 # Test your results with this code
18 ket_0 = np.array([1, 0])
19 ket_1 = np.array([0, 1])
20
21 print(f"<0|0> = {inner_product(ket_0, ket_0)}")
22 print(f"<0|1> = {inner_product(ket_0, ket_1)}")
23 print(f"<1|0> = {inner_product(ket_1, ket_0)}")
24 print(f"<1|1> = {inner_product(ket_1, ket_1)}")
25

```

CODERCISE 1.1.3 Sampling Measurement Outcomes

Write the function `measure_state` that takes a quantum state vector as input and simulates the outcomes of an arbitrary number of quantum measurements, i.e., return a list of samples 0 or 1 based on the probabilities given by the input state.

```

1 def measure_state(state, num_meas):
2     """Simulate a quantum measurement process.
3
4     Args:
5         state (np.array[complex]): A normalized qubit state vector.
6         num_meas (int): The number of measurements to take
7
8     Returns:
9         np.array[int]: A set of num_meas samples, 0 or 1, chosen according to the probability
10        distribution defined by the input state.
11    """
12
13    # Compute measurement outcome probabilities
14    probs = np.abs(state)**2
15
16    # Return a list of sample measurement outcomes
17    samples = np.random.choice([0, 1], size=num_meas, p=probs)
18
19    return samples
20

```

CODERCISE 1.1.4 Applying a Quantum Operation

Recall that quantum operations are represented as matrices. To preserve normalization, they must be a special type of matrix called a **unitary** matrix. For some 2×2 complex-valued unitary matrix U , the state of the qubit after an operation is

$$|\psi'\rangle = U|\psi\rangle.$$

Let's simulate the process by completing the function `apply_u` below to apply the provided quantum operation U to an input state.

```
1 U = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
2
3
4 def apply_u(state):
5     """Apply a quantum operation.
6
7     Args:
8         state (np.array[complex]): A normalized quantum state vector.
9
10    Returns:
11        np.array[complex]: The output state after applying U.
12    """
13
14    # APPLY U TO THE INPUT STATE AND RETURN THE NEW STATE
15    return np.dot(U, state)
16
```

CODERCISE 1.1.5 A Sample Quantum Algorithm

You may not have realized it, but you now have all the ingredients to write a very simple **quantum simulator** that can simulate the outcome of running quantum algorithms on a single qubit! Let's put everything together.

Use the functions below to simulate a quantum algorithm that does the following:

1. Initialize a qubit in state $|0\rangle$
2. Apply the provided operation U
3. Simulate measuring the output state 100 times

You'll have to complete a function for initialization, but we've provided functions for the other two.

```

1 U = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
2
3
4 def initialize_state():
5     """Prepare a qubit in state  $|0\rangle$ .
6
7     Returns:
8         np.array[float]: the vector representation of state  $|0\rangle$ .
9     """
10
11     # state  $|0\rangle$  is represented as the vector [1, 0]
12     return np.array([1.0, 0.0])
13
14
15 def apply_u(state):
16     """Apply a quantum operation."""
17     return np.dot(U, state)
18
19
20 def measure_state(state, num_meas):
21     """Measure a quantum state num_meas times."""
22     p_alpha = np.abs(state[0]) ** 2
23     p_beta = np.abs(state[1]) ** 2
24     meas_outcome = np.random.choice([0, 1], p=[p_alpha, p_beta], size=num_meas)
25     return meas_outcome
26
27
28 def quantum_algorithm():
29     """Use the functions above to implement the quantum algorithm described above.
30
31     Try and do so using three lines of code or less!
32
33     Returns:
34         np.array[int]: the measurement results after running the algorithm 100 times
35     """
36
37     # initialize the state
38     state = initialize_state()
39
40     # apply the unitary operation U
41     new_state = apply_u(state)
42
43     # take 100 measurement samples
44     return measure_state(new_state, 100)
45

```